

Digital Design 일반

🔴 동기 설계·FSM·파이프라이닝·reset 전략·합성을 아우르는 RTL 디지털 설계 기본기 · 📌 P1 · 🧠 메모리 브릿지: 메모리 컨트롤러의 command FSM·refresh 카운터·파이프라인, reset 도메인, Peri의 SRAM 컴파일러 사용이 전부 디지털 설계. "동작 RTL과 합성 후가 다를 수 있다"는 sign-off 마인드와 직결.

핵심 개념

- **동기 vs 비동기**: 동기=모든 FF 공통 clock, STA로 정적 검증 쉬움(주류). 비동기=글로벌 clock 없음, 빠르고 저전력이나 hazard·검증 난해.
- **FSM**: Moore=출력이 **현재 state만** 의존(registered, glitch 없음, 1-cycle latency). Mealy=출력이 state+입력 의존(빠른 반응, state 적음, but 입력 glitch가 출력에 전파).
- **Pipelining**: 조합 로직을 register로 stage 분할 → throughput(주파수)↑, latency는 같거나↑. 같은 일을 더 빠른 clock으로.
- **Hazard**: 구조적(자원 충돌), 데이터(**RAW** real, WAR/WAW), 제어(branch). 해결 =forwarding/stall/bubble, branch prediction.
- **Reset 전략**: async reset(언제든 assert, 단 **해제는 동기화** 필요=recovery/removal, 메타 위험→RDC). sync reset(clock edge에만, clean timing이나 running clock·충분한 reset 폭 필요). 표준=**reset synchronizer(async assert, sync deassert)**.
- **RDC(Reset Domain Crossing)**: 서로 다른 reset 도메인 간 경로. reset assert/deassert가 capture clock과 비동기면 메타 → CDC와 동일 원리로 검증.
- **합성**: RTL→gate netlist. **동작(시물) RTL ≠ 합성 후 동일 보장 아님**: incomplete sensitivity list, 의도치 않은 latch 추론, blocking/non-blocking 오용 → simulation-synthesis mismatch. 합성은 "기능 동일한 다른 구조"로 매핑.
- **Glitch**: 경로 지연차로 생기는 일시적 천이. 절대 raw 조합 출력으로 clock/reset/async 입력 쓰지 말 것.
- **Fan-out/buffering**: 한 드라이버가 많은 부하 → 지연·transition 악화 → buffer tree. max fanout/transition 제약.
- **SRAM/메모리 컴파일러**: 6T bitcell 기반, 컴파일러가 .lib/.lef 생성(표준셀 아님). Peri 로직이 둘러쌘.

예상 면접 Q&A

Q1. 동기 설계와 비동기 설계의 장단점은? 왜 동기가 주류인가요?

A. 동기 설계는 모든 FF가 같은 clock을 받아 STA로 setup/hold를 정적 검증할 수 있고 타이밍이 예측 가능해 검증·구현이 쉽습니다. 비동기는 글로벌 clock이 없어 잠재적으로 더 빠르고 저전력이지만, race·hazard가 많고 정적 타이밍 검증 도구가 약해 설계·검증이 어렵습니다. 그래서 상용 ASIC은 동기 설계가 주류이고, 도메인 경계만 CDC로 다룹니다. - L, 꼬리질문: 그럼 chip 안에 비동기 요소는 전혀 없나? → clock 도메인이 여러 개면 그 경계는 본질적으로 비동기 → CDC가 필요한 이유. - 🧠 메모리 브릿지: 메모리는 코어/PHY/관리 등 여러 동기 도메인 + 경계 CDC 조합.

Q2. Moore와 Mealy FSM의 차이를 설명하세요.

A. Moore FSM은 출력이 현재 state에만 의존해 출력이 register 출력처럼 안정적이고 glitch가 없지만, 입력 변화가 출력에 반영되는 데 한 clock 더 걸립니다. Mealy FSM은 출력이 state와 현재 입력 모두에 의존해 더 적은 state로 더 빠르게 반응하지만, 입력의 조합 경로가 출력에 직접 연결돼 glitch가 생길 수 있고 타이밍이 까다롭습니다. 보통 출력을 한 번 더 register해 Mealy의 빠름과 Moore의 안정성을 절충합니다(registered Mealy). - L, 꼬리질문: state encoding(binary/one-hot/gray) 선택 기준? → one-hot은 빠르고 디코딩 간단 (FPGA·고속), binary는 FF 적음, gray는 인접 전이 1비트(저전력·CDC 카운터). - 🧠 메모리 브릿지: 메모리 command 시퀀스(ACT→RD/WR→PRE)는 FSM, 타이밍 제약(tRCD 등) 카운터와 결합.

Q3. 파이프라이닝은 무엇을 개선하고 hazard에는 어떤 종류가 있나요?

A. 긴 조합 경로를 register로 여러 stage로 나눠 각 stage가 짧아지므로 clock 주파수=throughput이 올라갑니다. 한 데이터의 latency는 같거나 늘지만 단위시간당 처리량이 증가합니다. hazard는 (1)구조적: 같은 자원을 동시에 쓰려는 충돌, (2)데이터: 앞 명령 결과를 뒤가 필요로 하는 의존(주로 RAW), (3)제어: 분기로 다음 명령이 불확실. 해결은 forwarding(bypass), stall/bubble, branch prediction입니다. - L, 꼬리질문: stage를 무한히 늘리면? → register overhead($t_{cq} + \text{setup}$)와 hazard penalty가 늘어 오히려 성능·전력 손해. 최적 깊이가 있음. - 🧠 메모리 브릿지: 메모리 데이터패스/ECC/burst 처리도 파이프라인으로 대역폭 확보.

Q4. async reset과 sync reset의 차이, 그리고 reset synchronizer가 필요한 이유는?

A. async reset은 clock과 무관하게 즉시 assert돼 응급 초기화에 좋지만, 해제(deassert)가 clock edge 근처에서 일어나면 recovery/removal 위반→메타가 됩니다. sync reset은 clock edge에서만 동작해 타이밍이 깨끗하지만, running clock이 있어야 하고 reset 폭이 한 주기 이상이어야 합니다. 그래서 표준은 reset synchronizer: reset을 비동기로 assert하되 해제는 2-FF로 동기화해(async assert, sync deassert) 모든 FF가 같은 edge에 동시에 풀리게 합니다. - L, 꼬리질문: 이게 CDC와 무슨 관계? → reset도 신호라, 다른 reset/clock 도메인으로 넘어가면 RDC(Reset Domain Crossing)로 CDC와 똑같이 메타를 검증해야 함. - 🧠 메모리 브릿지: 메모리 IP의 다중 reset(power-on, soft, PHY)의 해제 정렬을 reset synchronizer·RDC로 sign-off.

Q5. "시뮬레이션이 통과한 RTL이 합성 후 다르게 동작"할 수 있는 이유는?

A. RTL은 동작을 기술하고 합성은 그것을 기능 동일한 gate로 매핑하지만, 코딩이 합성 규칙과 어긋나면 둘이 갈립니다. 대표적으로 (1)combinational always의 incomplete sensitivity list(시뮬은 일부만 반응, 합성은 전체로 봄), (2)의도치 않은 latch 추론(if/case에서 모든 경로에 대입 누락), (3)blocking(=)·non-blocking(<=) 오용으로 인한 race, (4) initial·delay 등 비합성 구문 사용입니다. 그래서 lint와 equivalence check(LEC)로 RTL↔netlist 동치를 정적 검증합니다. - L, 꼬리질문: 이것 막는 sign-off 단계는? → lint(코딩 규칙)+CDC+LEC. 저는 lint/CDC 방법론과 자동화를 담당해 이런 mismatch를 사전에 차단했습니다. - 🧠 메모리 브릿지: 메모리 Peri RTL도 동일 sign-off를 거쳐야 silicon 위험 제거.

Q6. glitch가 왜 위험하고 어디에 쓰면 안 되나요?

A. glitch는 조합 로직에서 입력들이 서로 다른 경로 지연으로 도착해 최종값으로 안정되기 전 잠깐 나타나는 잘못된 천이입니다. 동기 FF의 데이터 입력이면 clock edge에서 안정값만 캡처하니 무해하지만, clock·reset·async set/clear·CDC enable처럼 edge/level로 직접 동작하는 입력에 raw 조합 출력을 쓰면 가짜 트리거가 돼 치명적입니다. 그래서 gated clock은 ICG로, 조합 신호를 clock으로 쓰지 않습니다. - L, 꼬리질문:

glitch power도 문제? → 맞다. 불필요한 토글로 dynamic power 낭비 → path balancing. - 🧠 메모리 브릿지: clock/strobe 계열은 glitch-free가 필수 — 메모리 PHY clock 생성에 특히 엄격.

Q7. fan-out과 buffering, 그리고 메모리 컴파일러(SRAM) 기초를 설명하세요.

A. 한 게이트가 너무 많은 부하를 구동하면 출력 transition이 느려지고 지연이 커져 setup·signal integrity가 나빠집니다. 그래서 buffer/inverter tree로 부하를 분산하고 max fan-out·max transition 제약을 둡니다. 메모리는 표준셀로 만들지 않고 **메모리 컴파일러**가 원하는 깊이·폭의 SRAM(6T bitcell 기반)을 생성해 .lib(타이밍)·.lef(물리)·모델을 함께 내줍니다. 디지털 설계자는 그 hard macro를 인스턴스화하고 주변(peri) 로직·BIST를 붙입니다. - L, 꼬리질문: high fan-out net의 대표 예? → clock·reset·scan enable → 전용 tree(CTS)나 buffering으로 균형. - 🧠 메모리 브릿지: DRAM은 SRAM과 달리 1T1C cell·sense amp 기반이지만, "컴파일된 매크로+peri 디지털" 구도는 동일 — 신입의 메모리 갭을 메우는 핵심 그림.

30초 암기 요약

- 동기=공통 clock·STA 정적검증 쉬움(주류), 비동기=빠르나 hazard·검증 난해. 도메인 경계만 CDC.
- Moore=state만 의존(glitch 없음, 1cycle 느림), Mealy=state+입력(빠름·state 적음, glitch 위험). 절충 =registered Mealy.
- Pipelining=stage 분할로 throughput↑(latency는 유지/증가). hazard=구조/데이터(RAW)/제어 → forwarding·stall·예측.
- async reset=즉시 assert·해제 동기화 필요(recovery/removal), sync reset=edge에서만. 표준=reset synchronizer(async assert/sync deassert).
- reset도 도메인 넘으면 RDC → CDC와 동일 검증.
- 합성: 동작 RTL ≠ 합성 후 동일 보장 아님. incomplete sensitivity·latch 추론·blocking 오용 → lint+LEC로 차단.
- glitch는 clock/reset/CDC enable에 raw 조합 출력 쓰면 치명. gated clock은 ICG.
- 메모리=메모리 컴파일러가 SRAM 매크로(.lib/.lef) 생성, peri 디지털이 둘러쌘.